

## **Informe Videojuego**

**Nicolas Trigos Morales:** [ntrigosm@unal.edu.co](mailto:ntrigosm@unal.edu.co)

**Juan Lorenzo Barajas Moya:** [jbarajasm@unal.edu.co](mailto:jbarajasm@unal.edu.co)

**Luis Alejandro Caceres Olejua:** [lcacereso@unal.edu.co](mailto:lcacereso@unal.edu.co)

**Profesor: Rafael Antonio Acosta Rodriguez**

**Universidad Nacional de Colombia**

**Facultad de Ingeniería**

**Programación Orientada a Objetos ( POO)**

**2025**

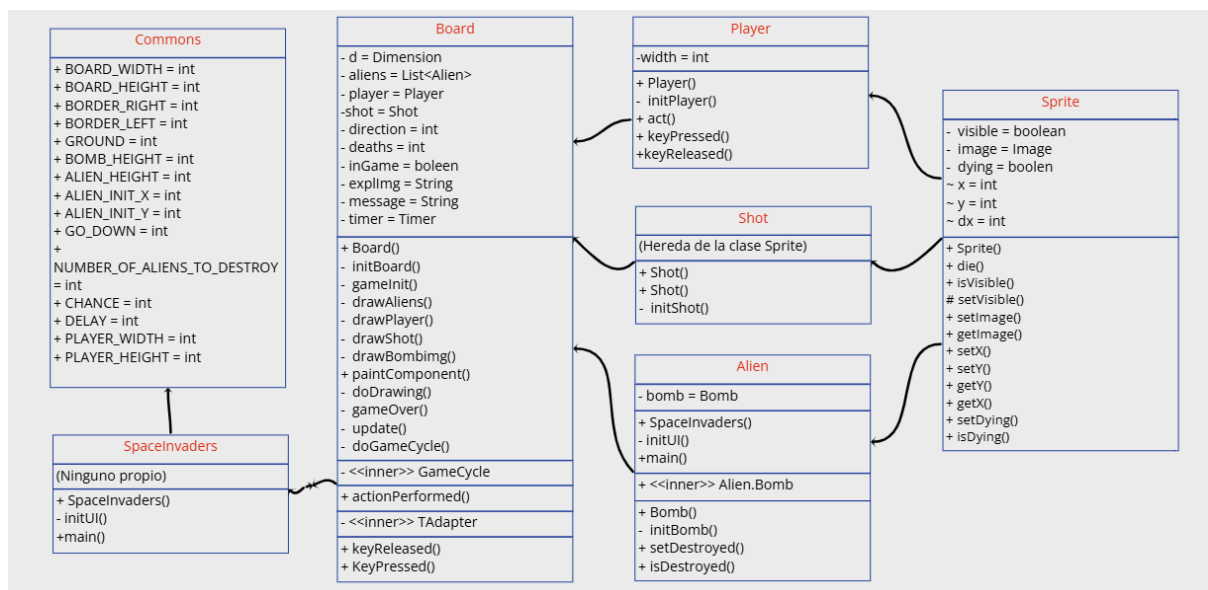
## Resumen

En este informe se abordan aspectos como la herencia, el polimorfismo, el encapsulamiento, las clases, los atributos y el diagrama UML que conforman el juego Space Invaders.

## Introducción

En este informe se presenta un análisis detallado del código del juego Space Invaders, siguiendo la temática de los juegos de los años 80 propuesta por el docente. El código está desarrollado en el lenguaje de programación Java, y su objetivo es identificar los diferentes componentes que lo conforman, tales como la herencia, el polimorfismo, el encapsulamiento, las clases y los atributos. Además, se elaboran diagramas UML de estos elementos, con el propósito de comprender con mayor profundidad las capacidades y el uso que se le puede dar al lenguaje Java.

## Diagrama UML



## 1. Clase SpaceInvaders

La clase `SpaceInvaders` es la main class, se encarga de configurar la interfaz, inicializa el tablero del juego (`Board`) y se lanza la aplicación en el hilo gráfico. Es decir, esta clase no maneja la lógica del juego, solo prepara el entorno visual y el marco donde el juego ocurre.

```
package com.zetcode;

import java.awt.EventQueue;
import javax.swing.JFrame;
```

`EventQueue` para ejecutar la interfaz gráfica de forma segura.  
`JFrame`, que es la ventana base en `Swing`.

```
public class SpaceInvaders extends JFrame {
```

Hereda de `JFrame`, por lo que `SpaceInvaders` es una ventana.  
 Esto permite definir su tamaño, título, comportamiento al cerrar, etc.

```
public class SpaceInvaders extends JFrame {

    public SpaceInvaders() {

        initUI();

    }
}
```

Este es el constructor del juego.

Al crear un objeto `SpaceInvaders`, automáticamente llama a `initUI()`.

Su propósito: construir la interfaz del juego (UI = User Interface).

```
private void initUI() {

    add(new Board());

    setTitle("Space Invaders");
    setSize(Commons.BOARD_WIDTH, Commons.BOARD_HEIGHT);

    setDefaultCloseOperation(EXIT_ON_CLOSE);
    setResizable(false);
    setLocationRelativeTo(null);

}
```

Añade el panel principal del juego. `Board()` es donde se dibujan los enemigos, el jugador, los disparos, etc. En el método se definen parámetros como el título de la ventana, el tamaño, la acción de cerrar la aplicación al salir, el bloqueo de reescalado y el centrado

```
public static void main(String[] args) {

    EventQueue.invokeLater(() -> {

        var ex = new SpaceInvaders();
        ex.setVisible(true);

    });

}
```

Este es el método main Es el punto de entrada del programa.

Usa `EventQueue.invokeLater()` para que la interfaz se cree en el hilo gráfico de Swing (Event Dispatch Thread), evitando errores de concurrencia.

Dentro del `invokeLater`:

Se crea una instancia de `SpaceInvaders` → esto ejecuta el constructor y llama `initUI()`.

Luego `ex.setVisible(true)` muestra la ventana al jugador.

## 2. Clase Board

```
public class Board extends JPanel {

    private Dimension d;
    private List<Alien> aliens;
    private Player player;
    private Shot shot;

    private int direction = -1;
    private int deaths = 0;

    private boolean inGame = true;
    private String explosion = "src/main/java/images/explosion.png";
    private String message = "Game Over";

    private Timer timer;
```

`d` → tamaño del tablero.

`aliens` → lista de enemigos activos.

`player` → el jugador controlado por el teclado.

`shot` → el disparo actual del jugador.

`direction` → dirección de movimiento de los aliens (1 = derecha, -1 = izquierda).

`deaths` → cantidad de enemigos destruidos.

`inGame` → indica si el juego sigue o si terminó.

`timer` → controla el bucle del juego (actualiza y repinta cada cierto tiempo).

```
public Board() {
    initBoard();
    gameInit();
}
```

Inicializa el tablero (`initBoard()`) y los objetos del juego (`gameInit()`).

```
private void initBoard() {
    addKeyListener(new TAdapter());
    setFocusable(true);
    d = new Dimension(Commons.BOARD_WIDTH, Commons.BOARD_HEIGHT);
    setBackground(Color.black);

    timer = new Timer(Commons.DELAY, new GameCycle());
    timer.start();

    gameInit();
}
```

El método `initBoard()` captura el teclado con `TAdapter` (una clase interna).

Define el tamaño y el color del fondo.

Inicia el temporizador (`Timer`), que llama repetidamente a `GameCycle` cada cierto tiempo (determinado por `Commons.DELAY`).

Eso crea el bucle principal del juego, que constantemente actualiza la lógica y redibuja la pantalla.

```
private void gameInit() {
    aliens = new ArrayList<>();

    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 6; j++) {
            var alien = new Alien(Commons.ALIEN_INIT_X + 18 * j,
                                   Commons.ALIEN_INIT_Y + 18 * i);
            aliens.add(alien);
        }
    }

    player = new Player();
    shot = new Shot();
}
```

Inicializa los personajes:

Crea una matriz de 4 filas por 6 columnas de aliens.

Crea al jugador.

Crea un disparo vacío (que se activará luego).

### Dibujo del juego

Los métodos `drawAliens()`, `drawPlayer()`, `drawShot()` y `drawBombing()` dibujan los distintos elementos en pantalla si están visibles.

`paintComponent(Graphics g)` llama a `doDrawing(g)` cada vez que el panel se repinta:

```
private void doDrawing(Graphics g) {

    g.setColor(Color.black);
    g.fillRect(0, 0, d.width, d.height);
    g.setColor(Color.green);

    if (inGame) {

        g.drawLine(0, Commons.GROUND,
                   Commons.BOARD_WIDTH, Commons.GROUND);

        drawAliens(g);
        drawPlayer(g);
        drawShot(g);
        drawBombing(g);

    } else {

        if (timer.isRunning()) {
            timer.stop();
        }

        gameOver(g);

    }

    Toolkit.getDefaultToolkit().sync();
}
```

Si el juego sigue, dibuja todos los objetos.

Si terminó, muestra la pantalla de “Game Over” o “Game Won”.

### Método update()

```
private void update()
```

Este es el cerebro del juego. Se ejecuta en cada ciclo del Timer.

Aquí:

Se mueve al jugador (`player.act()`).

Se mueve el disparo (`shot`).

Se revisan colisiones entre disparos y aliens.

Se cambia la dirección de los aliens cuando tocan los bordes.

Los aliens bajan una línea.

Se lanzan bombas aleatoriamente desde los aliens.

Se revisan colisiones entre bombas y el jugador.

Se verifica si el jugador ganó o perdió.

### Bucle de juego (GameCycle)

```
private class GameCycle implements ActionListener {

    @Override
    public void actionPerformed(ActionEvent e) {

        doGameCycle();

    }

}
```

El Timer llama a este método cada pocos milisegundos:

```
private void doGameCycle() {

    update();
    repaint();

}
```

Esto crea el bucle principal del juego:

Actualizar → Dibujar → Actualizar → Dibujar...

```
private class TAdapter extends KeyAdapter {

    @Override
    public void keyReleased(KeyEvent e) {

        player.keyReleased(e);

    }

    @Override
    public void keyPressed(KeyEvent e) {

        player.keyPressed(e);

        int x = player.getX();
        int y = player.getY();

        int key = e.getKeyCode();

        if (key == KeyEvent.VK_SPACE) {

            if (inGame) {

                if (!shot.isVisible()) {

                    shot = new Shot(x, y);

                }

            }

        }

    }

}
```

Captura las teclas para mover al jugador o disparar.  
Si se presiona espacio, se dispara un proyectil si no hay otro visible.

### 3. Clase commons

Esta clase cumple un papel muy importante en el diseño del juego:  
no tiene comportamiento (no hay métodos), sino que sirve como un repositorio central de constantes que definen las reglas y dimensiones del juego.

```
package com.zetcode;

public interface Commons {

    int BOARD_WIDTH = 358;
    int BOARD_HEIGHT = 350;
    int BORDER_RIGHT = 30;
    int BORDER_LEFT = 5;

    int GROUND = 290;
    int BOMB_HEIGHT = 5;

    int ALIEN_HEIGHT = 12;
    int ALIEN_WIDTH = 12;
    int ALIEN_INIT_X = 150;
    int ALIEN_INIT_Y = 5;

    int GO_DOWN = 15;
    int NUMBER_OF_ALIENS_TO_DESTROY = 24;
    int CHANCE = 5;
    int DELAY = 17;
    int PLAYER_WIDTH = 15;
    int PLAYER_HEIGHT = 10;
}
```

En Java, todas las variables definidas en una interfaz son automáticamente:

- public (públicas),
- static (compartidas por todas las clases),
- final (constantes, no se pueden modificar).

Eso significa que cualquier clase del juego puede acceder a ellas directamente.

### 4. Clase Alien

En esta clase se definirá el funcionamiento del alien, es decir, su posición inicial, la inicialización de la bomba (ataque al jugador) y sus desplazamientos.



En la siguiente imagen podemos ver como se le da un valor inicial de posicionamiento al alien y se inicializa la bomba que lanzará este último. Además se le asigna la correspondiente al alien.

```
public Alien(int x, int y) {
    initAlien(x, y);
}

private void initAlien(int x, int y) {
    this.x = x;
    this.y = y;

    bomb = new Bomb(x, y);

    var alienImg = "src/images/alien.png";
    var ii = new ImageIcon(alienImg);

    setImage(ii.getImage());
}
```

Ahora vemos este comando, que actualiza la posición del alien según las indicaciones del juego.

```
public void act(int direction) {
    this.x += direction;
}
```

Siguiendo con esta clase, vemos como el código define cuando la bomba va a ser usada o no, dándole una posición inicial cuando es destruida, pero siendo lanzada únicamente cuando el juego cambia la propiedad del objeto a false, es decir, se cambia el valor booleano que define si la bomba está destruida (lo que significa que no debe estar en pantalla) únicamente cuando la programación del juego lo permita, pero en esta clase se define su posición cuando la bomba está inactiva.

```
public class Bomb extends Sprite {
    private boolean destroyed;

    public Bomb(int x, int y) {
        initBomb(x, y);
    }

    private void initBomb(int x, int y) {
        setDestroyed(true);

        this.x = x;
        this.y = y;

        var bombImg = "src/images/bomb.png";
        var ii = new ImageIcon(bombImg);
        setImage(ii.getImage());
    }

    public void setDestroyed(boolean destroyed) {
        this.destroyed = destroyed;
    }

    public boolean isDestroyed() {
        return destroyed;
    }
}
```

## 5. Clase Player

En esta clase está determinada la movilidad y uso del personaje controlado por el usuario.

En esta imagen podemos contemplar la programación inicial del personaje, es decir, su posición inicial.

```
private void initPlayer() {

    var playerImg = "src/images/player.png";
    var ii = new ImageIcon(playerImg);

    width = ii.getImage().getWidth(null);
    setImage(ii.getImage());

    int START_X = 270;
    setX(START_X);

    int START_Y = 280;
    setY(START_Y);
}
```

Aquí, mientras tanto, vemos como se le suma o se le resta a la posición inicial del personaje según la tecla oprimida, donde se le suma 2 a la izquierda o derecha si se tiene alguna de estas dos teclas presionadas, o se detiene si se sueltan.

```
public void keyPressed(KeyEvent e) {

    int key = e.getKeyCode();

    if (key == KeyEvent.VK_LEFT) {

        dx = -2;
    }

    if (key == KeyEvent.VK_RIGHT) {

        dx = 2;
    }
}

public void keyReleased(KeyEvent e) {

    int key = e.getKeyCode();

    if (key == KeyEvent.VK_LEFT) {

        dx = 0;
    }

    if (key == KeyEvent.VK_RIGHT) {

        dx = 0;
    }
}
```

## 6. Clase Shot

Se revisará ahora cómo funciona el disparo del jugador.

En esta primera imagen vemos como el disparo toma la posición inicial del jugador para después ser lanzado.

```
public Shot(int x, int y) {

    initShot(x, y);
}
```

Ahora vemos el comando que se usa para que el disparo aparente salir del centro de la nave del jugador, además de que se le asigna una imagen al disparo

```
private void initShot(int x, int y) {

    var shotImg = "src/images/shot.png";
    var ii = new ImageIcon(shotImg);
    setImage(ii.getImage());

    int H_SPACE = 6;
    setX(x + H_SPACE);

    int V_SPACE = 1;
    setY(y - V_SPACE);
}
```

## 7. Clase Sprite

Esta clase es la responsable de todo aquello que lleve movilidad en el juego. Proporciona las propiedades de cualquier elemento gráfico.

Ahora veremos cómo se establecen los valores de cada objeto, sea su imagen, su posición inicial, y si el objeto en cuestión es visible o no.

```
public void setImage(Image image) {
    this.image = image;
}

public Image getImage() {
    return image;
}

public void setX(int x) {
    this.x = x;
}

public void setY(int y) {
    this.y = y;
}

public int getY() {
    return y;
}

public int getX() {
    return x;
}
```

## Conclusión

Se puede concluir que este informe nos ayudo a entender y comprender de una mejor manera la estructura de JAVA con el polimorfismo y más específicamente herencia, el cual fue usado constantemente en el desarrollo de este juego.

## Referencia

<https://github.com/janbodnar/Java-Space-Invaders/tree/master>